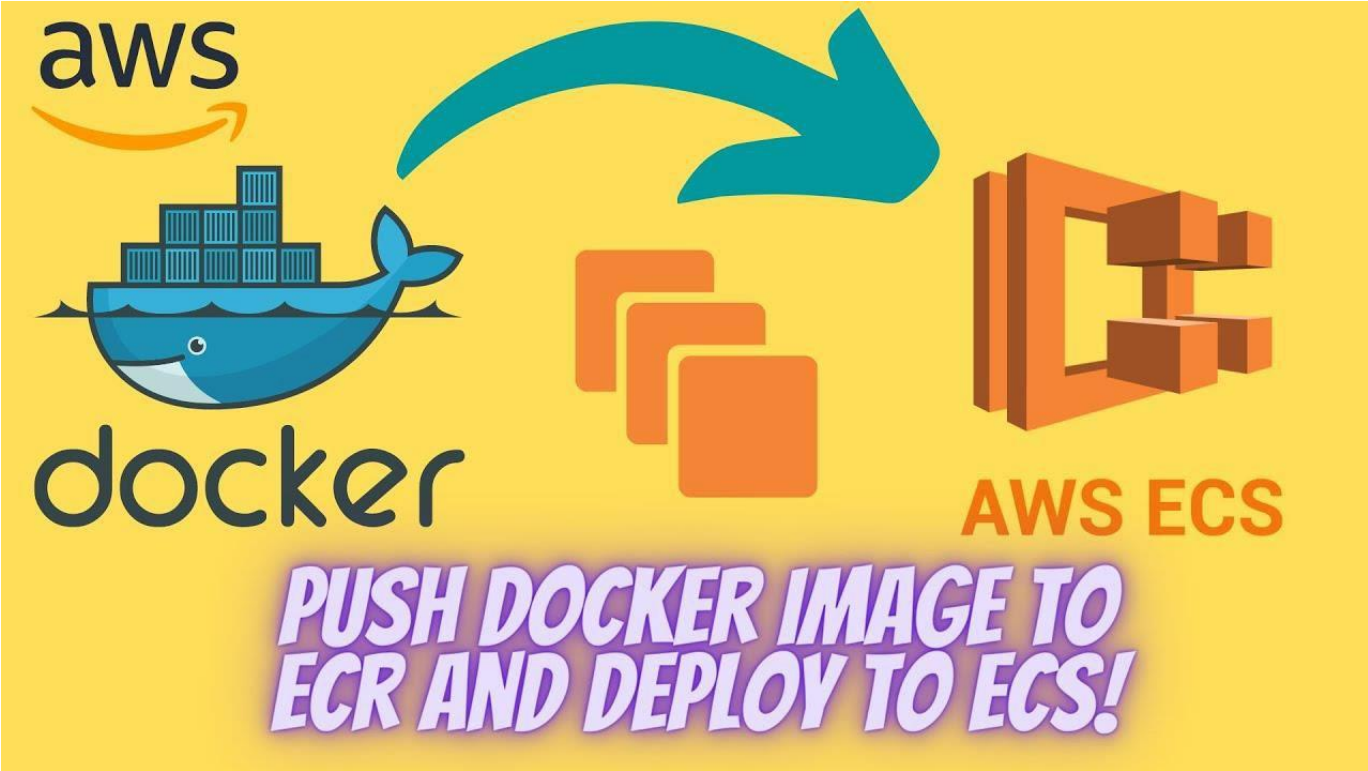
DEPLOYING A CONTAINER USING AWS ECS AND ECR



ECR:-

Amazon Elastic Container Registry (Amazon ECR) is an AWS managed container image registry service that is secure, scalable, and reliable. Amazon ECR supports private repositories with resource-based permissions using AWS IAM. This is so that specified users or Amazon EC2 instances can access your container repositories and images. You can use your preferred CLI to push, pull, and manage Docker images, Open Container Initiative (OCI) images, and OCI compatible artifacts.

ECS:-

Amazon Elastic Container Service (ECS) is a fully managed container orchestration service that helps you to more efficiently deploy, manage, and scale containerized applications. It deeply integrates with the AWS environment to provide an easy-to-use solution for running container workloads in the cloud and on premises with advanced security features using Amazon ECS Anywhere.

PRATHAMESH MISAL

Step 1:- Download or install 'aws cli' on your machine to access the aws resources through CLI.

Use the following commands :-

curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"

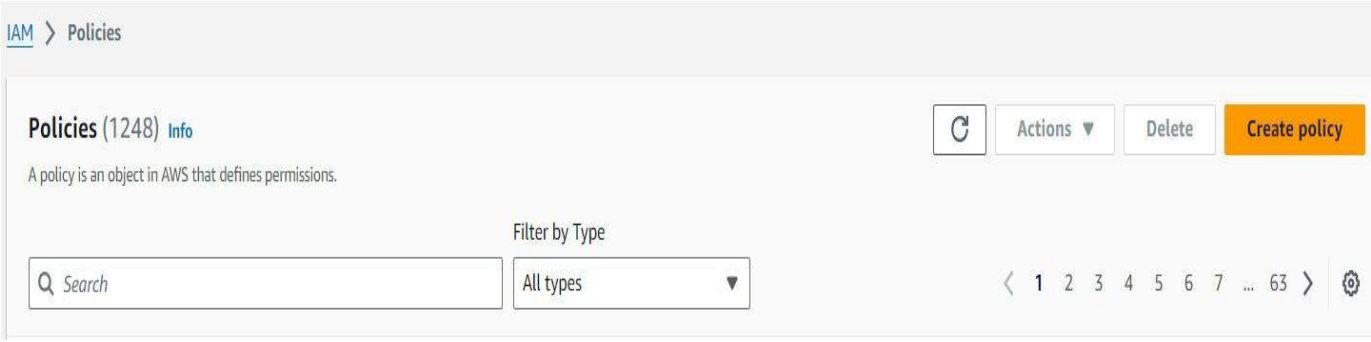
unzip awscliv2.zip sudo ./aws/install

Step 2:- Create a Dockerfile .

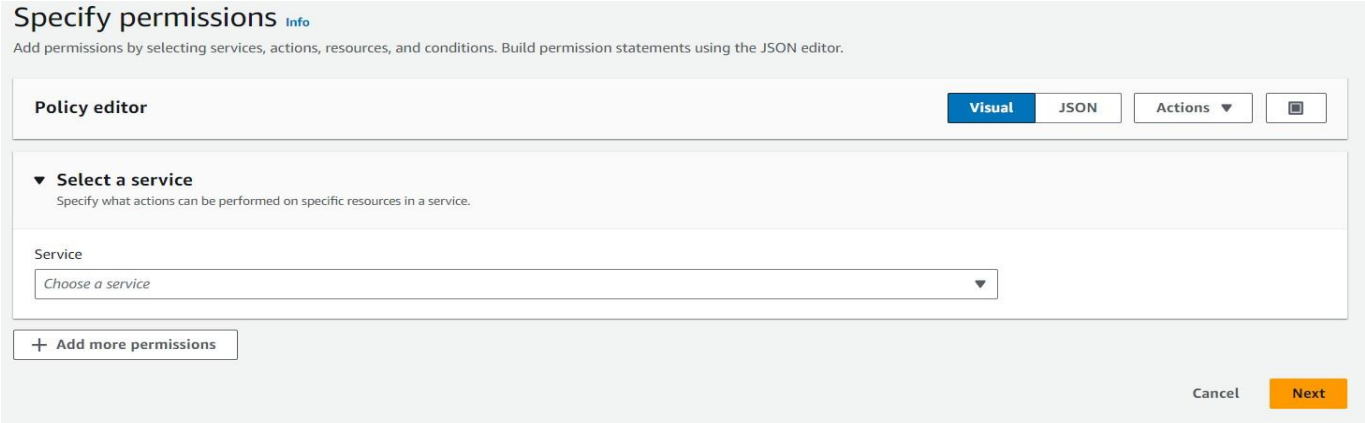
```
FROM docker.io/ubuntu
RUN apt update -y
RUN apt install apache2 -y
RUN echo "HEllO FROM PRATHAMESH" > /var/www/html/index.html
CMD ["apachectl","-D","FOREGROUND"]
~
~
```

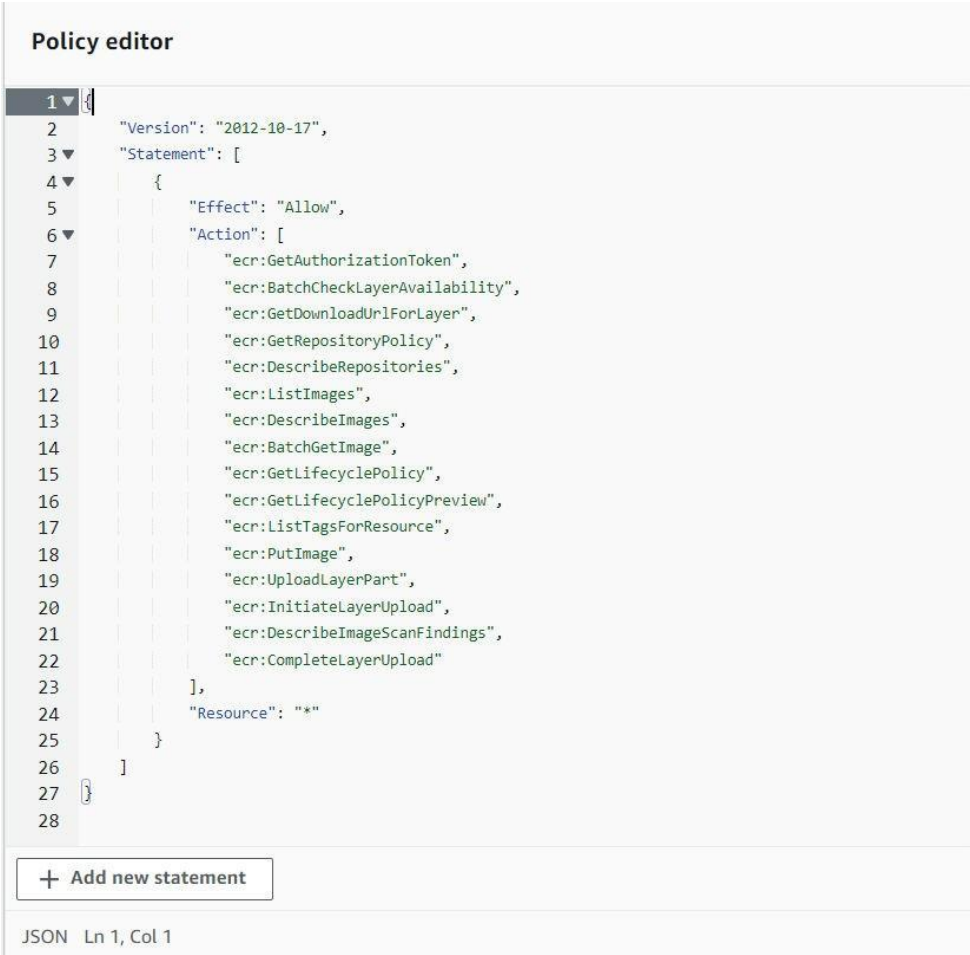
Step 3:- Create an IAM policy for ECR Access.

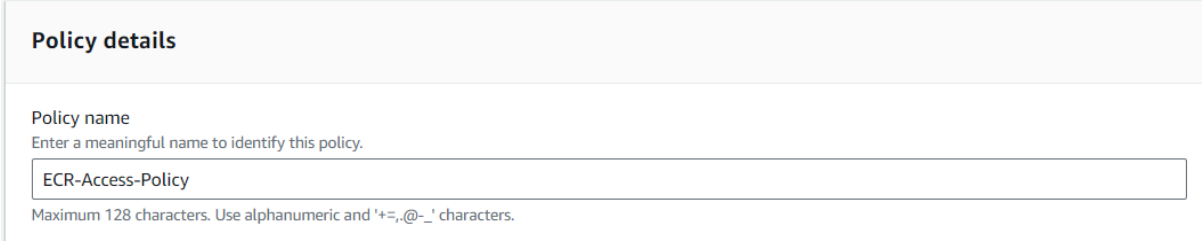
- Click on create policy button

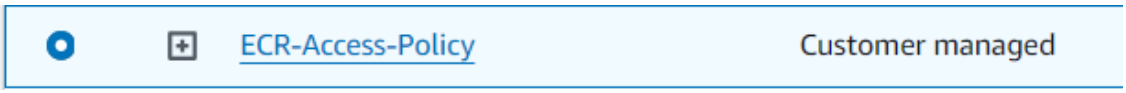


- Click on JSON



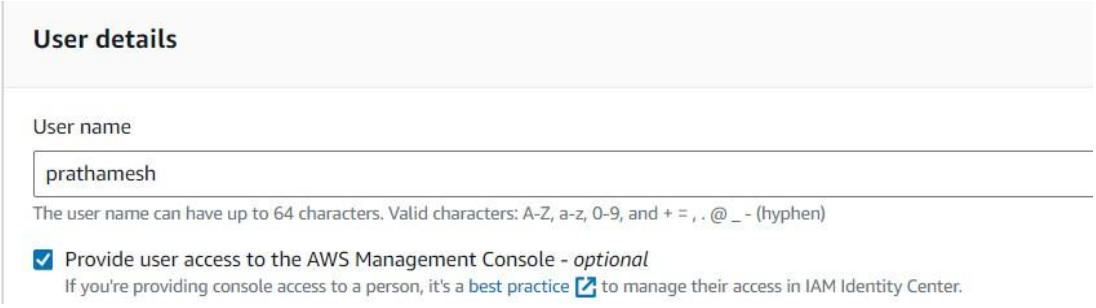




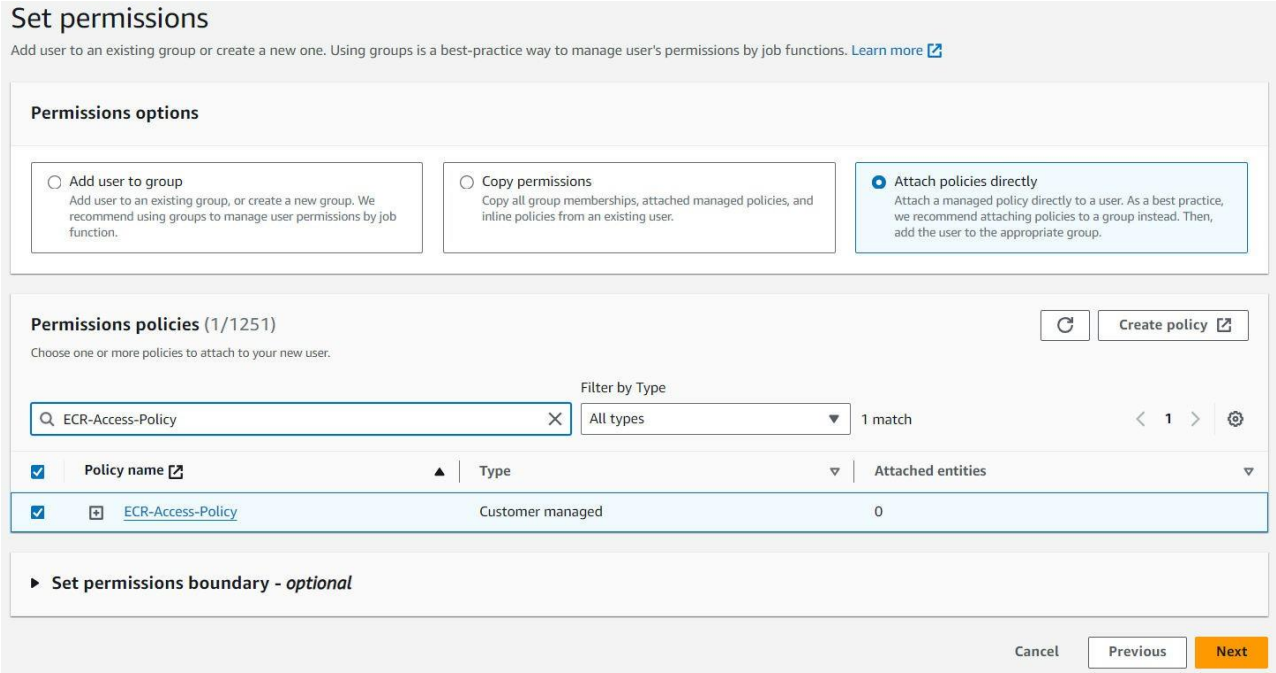


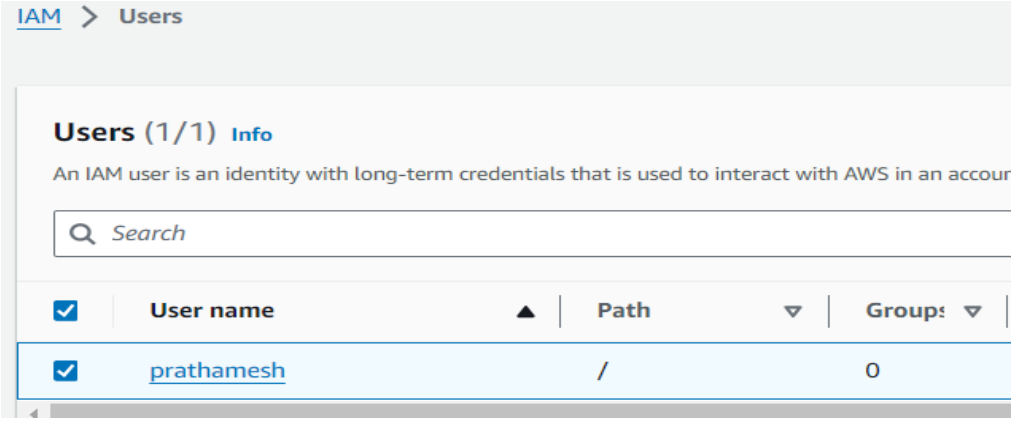
Step 4:- Create a IAM user and attach the above policy to it.

- Go to IAM Management Console
- Navigate to users section
- Click on Create User
- Specify Username



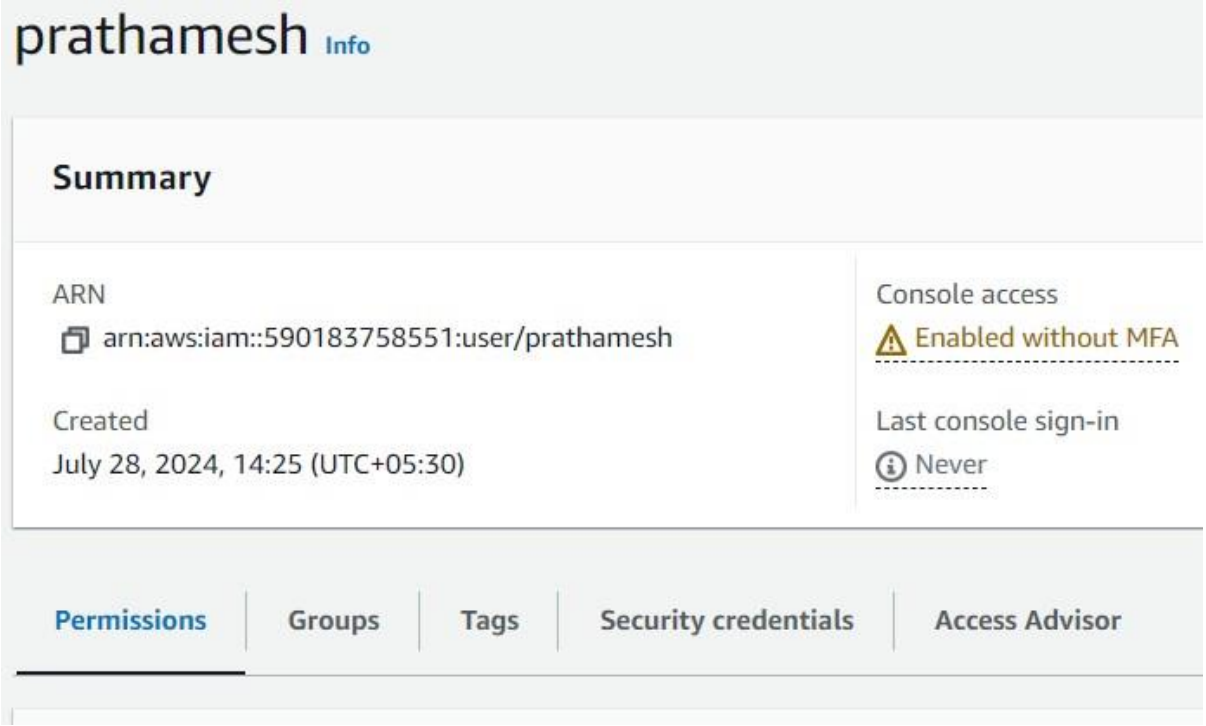
- Attach the policy that we created.



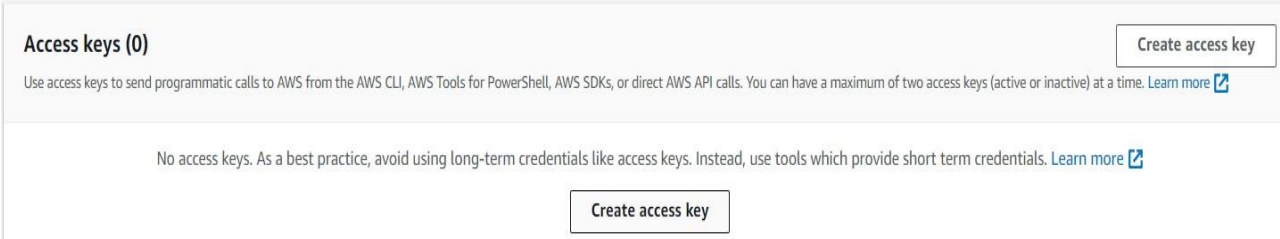


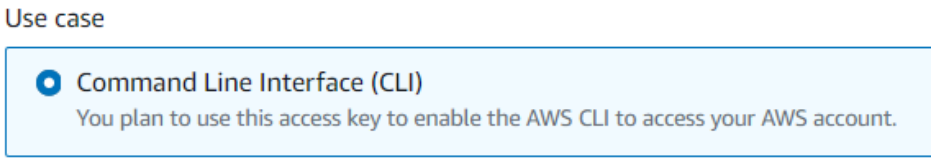
Step 5:- Now create Access key and secrete access key of the user which is required to configure the user on our machine.

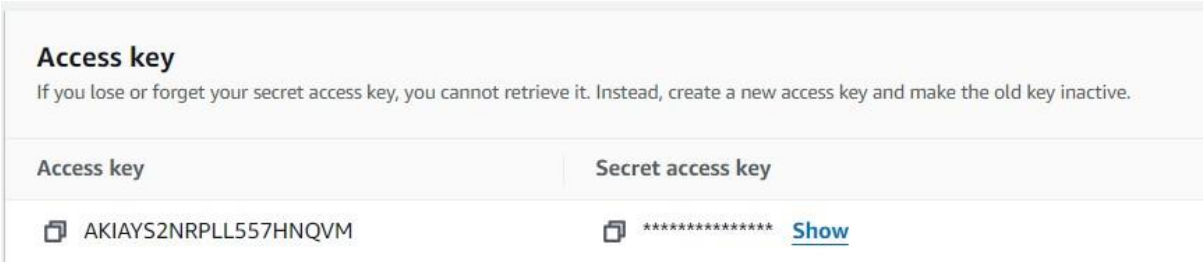
•Click on security credentials



- Click on create access key







Step 6:- Configure the user on our machine

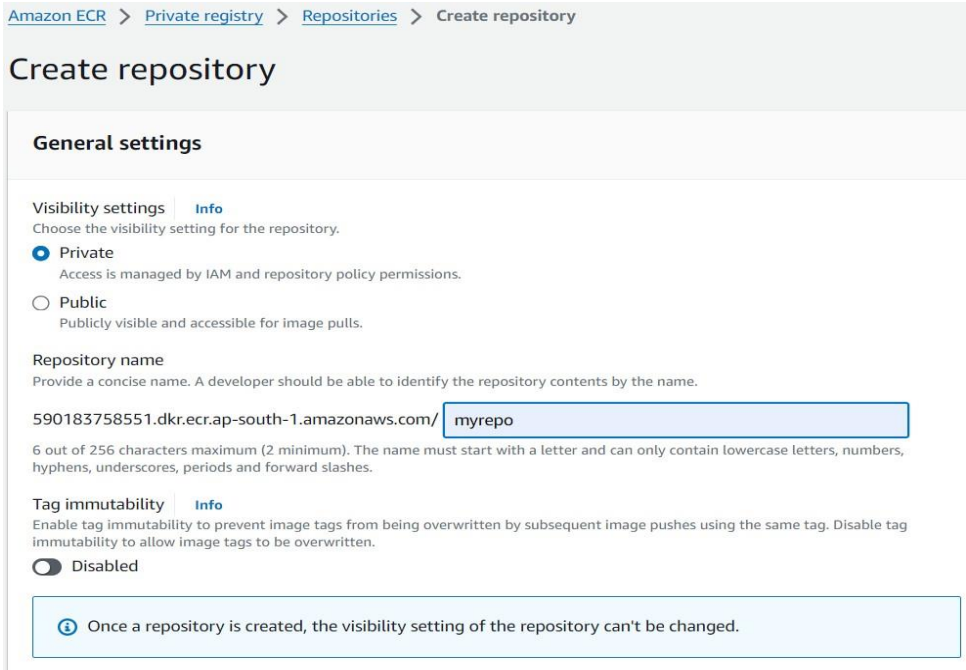
Cmd :- aws configure

```
ubuntu $ aws configure
AWS Access Key ID [None]: AKIAYS2NRPLL557HN
AWS Secret Access Key [None]: JFZrUNBNl6JY7
Default region name [None]:
Default output format [None]:
ubuntu $
```

Step 7:- Create an ECR Repository in AWS.

- Go to Amazon ECR service

- Click on create and give the repository name and also mention visibility private or public.



Private repositories

Improved basic scanning
Switch to the improved version of basic scanning, powered by AWS native scanning technology. Learn more.

Switch

Repositories (1)

View push commands | Delete | Actions | Create repository

Filter status

1

Repository name	URI	Created at	Tag immutability	Scan frequency	Encryption type
myrepo	590183758551.dkr.ecr.ap-south-1.amazonaws.com/myrepo	July 28, 2024, 14:39:08 (UTC+05.5)	Disabled	Manual	AES-256

Step 8:- Push the Docker image to ECR .

- Go inside the repo.
- Click on view Push commands.

Amazon ECR > Private registry > Repositories > myrepo

myrepo

View push commands

Push commands for myrepo

macOS / Linux | Windows

Make sure that you have the latest version of the AWS CLI and Docker installed. For more information, see Getting Started with Amazon ECR.

Use the following steps to authenticate and push an image to your repository. For additional registry authentication methods, including the Amazon ECR credential helper, see Registry Authentication.

1. Retrieve an authentication token and authenticate your Docker client to your registry. Use the AWS CLI:

```
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 590183758551.dkr.ecr.ap-south-1.amazonaws.com
```

Note: If you receive an error using the AWS CLI, make sure that you have the latest version of the AWS CLI and Docker installed.

2. Build your Docker image using the following command. For information on building a Docker file from scratch see the instructions here. You can skip this step if your image is already built:

```
docker build -t myrepo .
```

3. After the build completes, tag your image so you can push the image to this repository:

```
docker tag myrepo:latest 590183758551.dkr.ecr.ap-south-1.amazonaws.com/myrepo:latest
```

4. Run the following command to push this image to your newly created AWS repository:

```
docker push 590183758551.dkr.ecr.ap-south-1.amazonaws.com/myrepo:latest
```

Close

Step 9:- Run the push commands simultaneously.

- Authenticate Docker client to ECR

```
ubuntu $ aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 590183758551.dkr.ecr.ap-south-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
ubuntu $
```

- Build the Docker image from the dockerfile that we created above.

```
ubuntu $ docker build -t myrepo .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  278.1MB
Step 1/5 : FROM docker.io/ubuntu
latest: Pulling from library/ubuntu
9c704ecd0c69: Pull complete
Digest: sha256:2e863c44b718727c860746568e1d54afd13b2fa71b160f5cd9058fc436217b30
Status: Downloaded newer image for ubuntu:latest
 ---> 35a88802559d
Step 2/5 : RUN apt update -y
 ---> Running in 3c3a8a9ae828
```

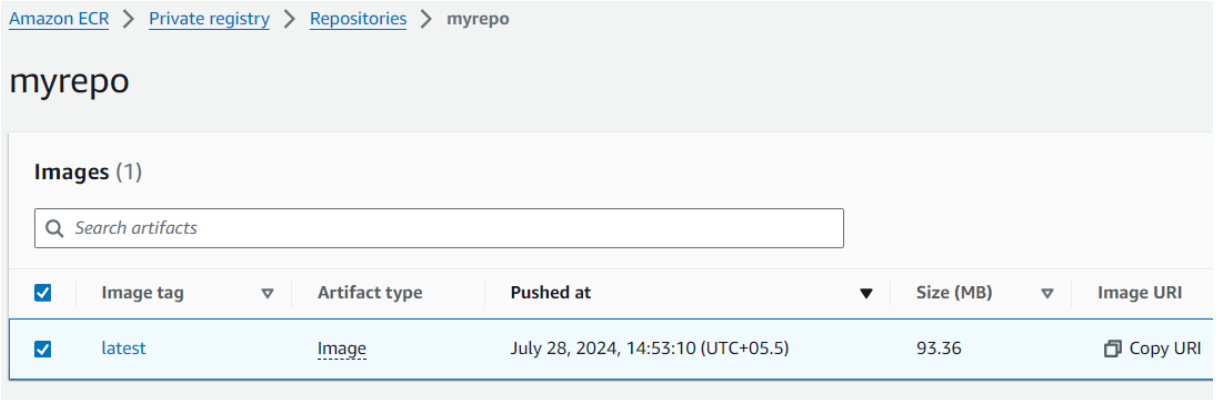
- Tag an image to push to our registry

```
ubuntu $ docker tag myrepo:latest 590183758551.dkr.ecr.ap-south-1.amazonaws.com/myrepo:latest
ubuntu $ docker images
REPOSITORY                                          TAG       IMAGE ID       CREATED          SIZE
590183758551.dkr.ecr.ap-south-1.amazonaws.com/myrepo  latest    39d6dd4e9b61   35 seconds ago   224MB
myrepo                                              latest    39d6dd4e9b61   35 seconds ago   224MB
ubuntu                                              latest    35a88802559d   7 weeks ago      78.1MB
ubuntu $
```

- Run the following command to push the image to ECR

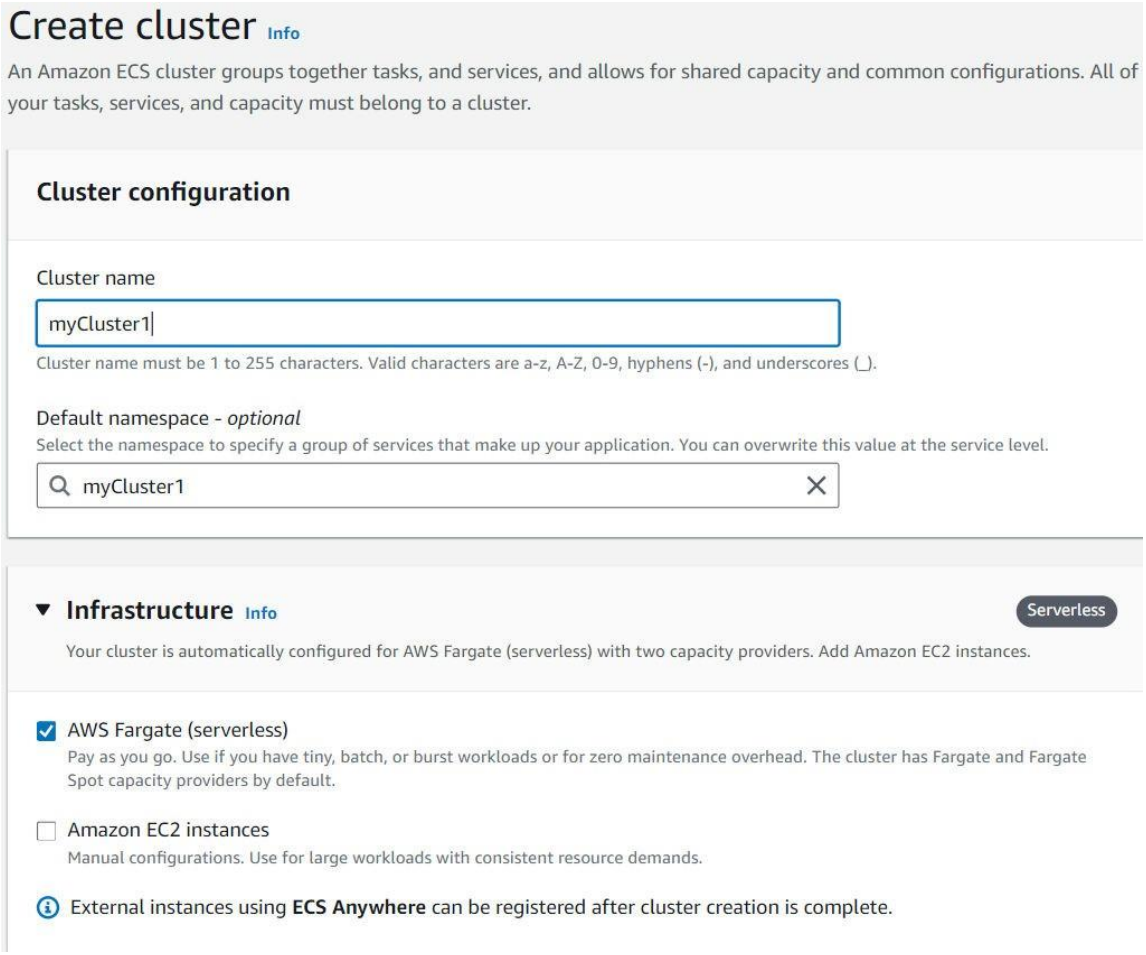
```
ubuntu $ docker push 590183758551.dkr.ecr.ap-south-1.amazonaws.com/myrepo:latest
The push refers to repository [590183758551.dkr.ecr.ap-south-1.amazonaws.com/myrepo]
5cf0f88c4684: Pushed
c37dfe62042e: Pushed
36d069da92aa: Pushed
a30a5965a4f7: Pushed
latest: digest: sha256:fb02aaafaf636071065aa5a55307224dfcc7a65763a617f8b7e1e7bec4c6a792 size: 1160
ubuntu $
```

Step 10:- We can see our image is successfully pushed to our ECR registry.



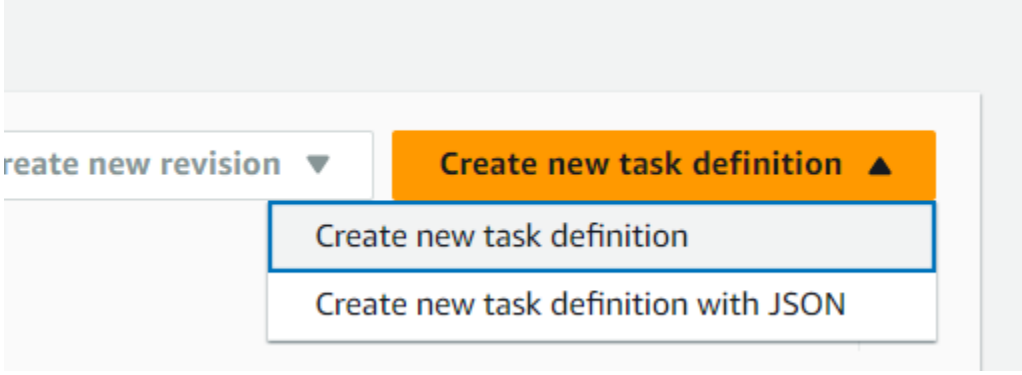
Step 11:- Navigate to ECS service

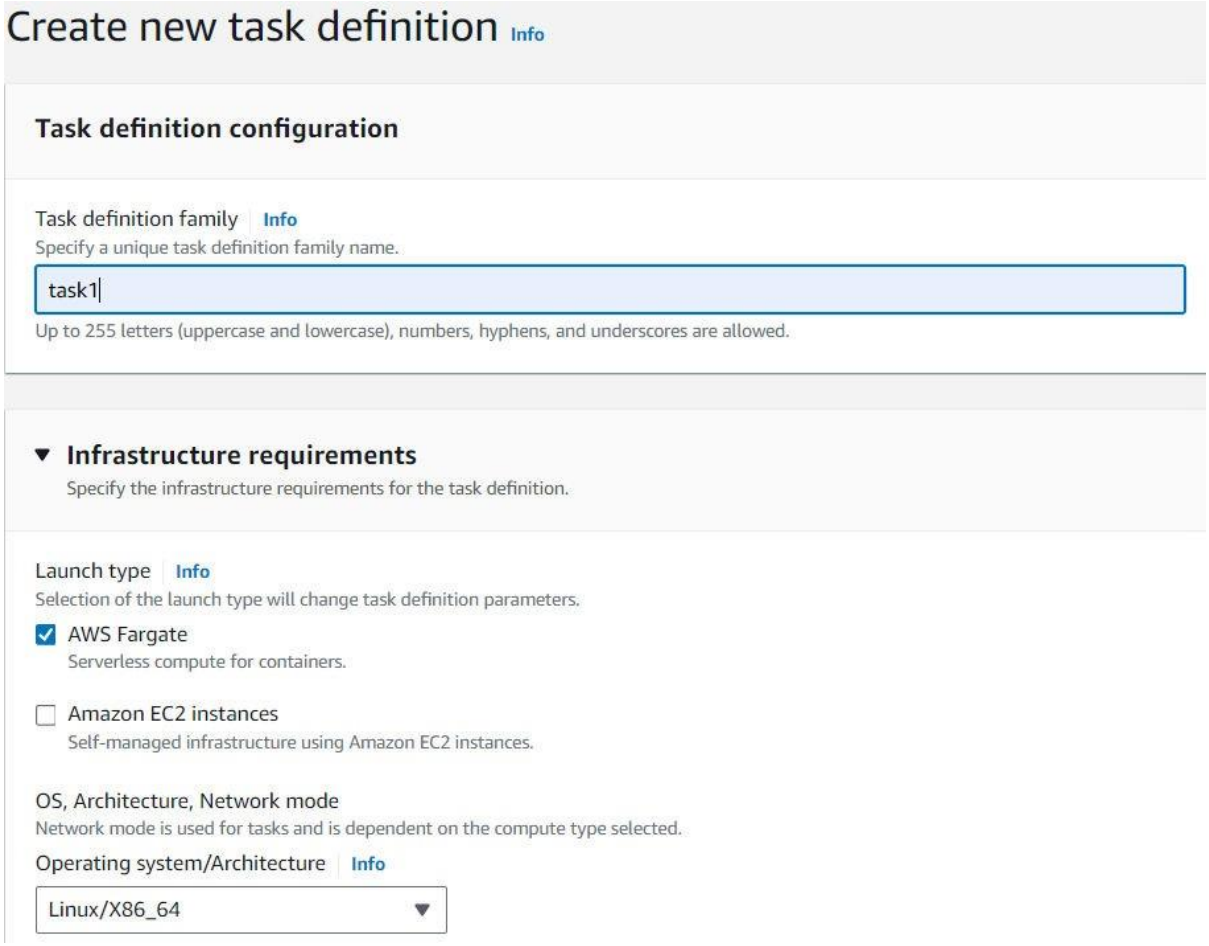
- Create a Cluster.
- Enter cluster name
- Choose "**AWS Fargate**" for serverless deployment



Step 12:- Create a task Definition

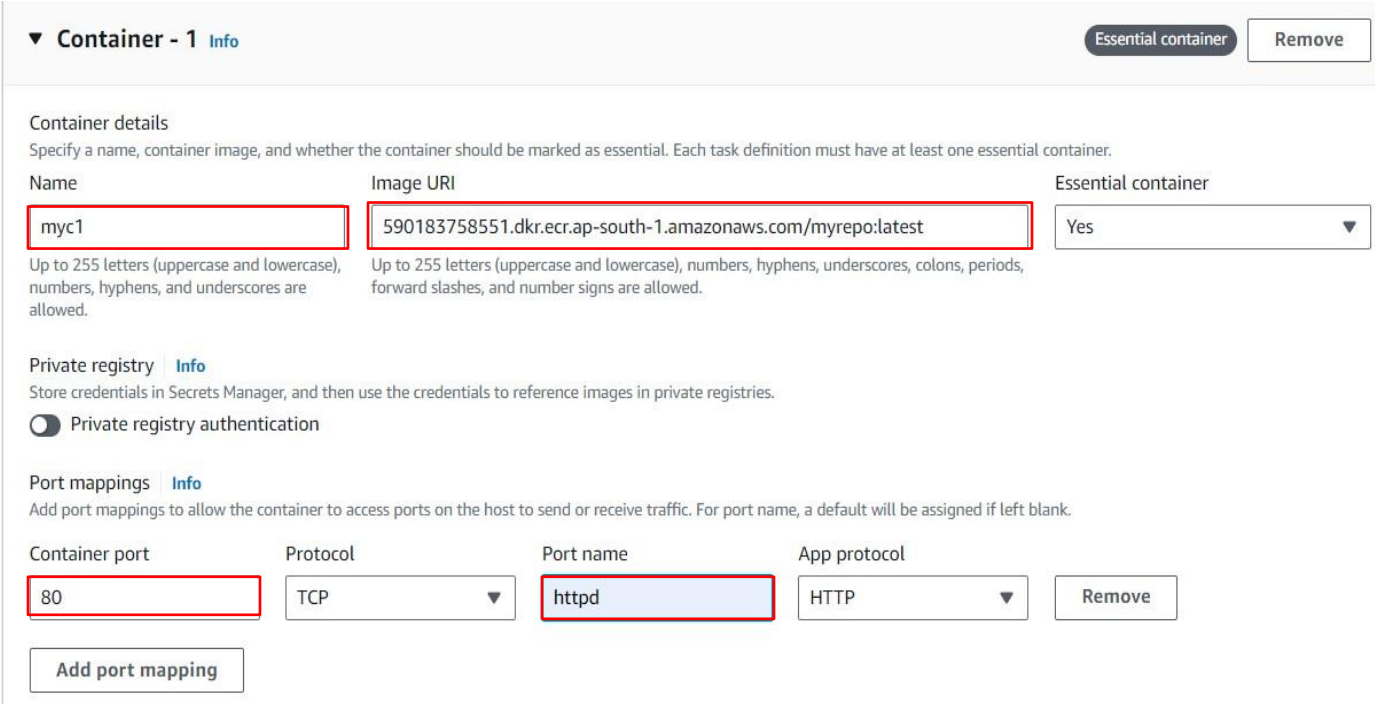
- Create a new task definition



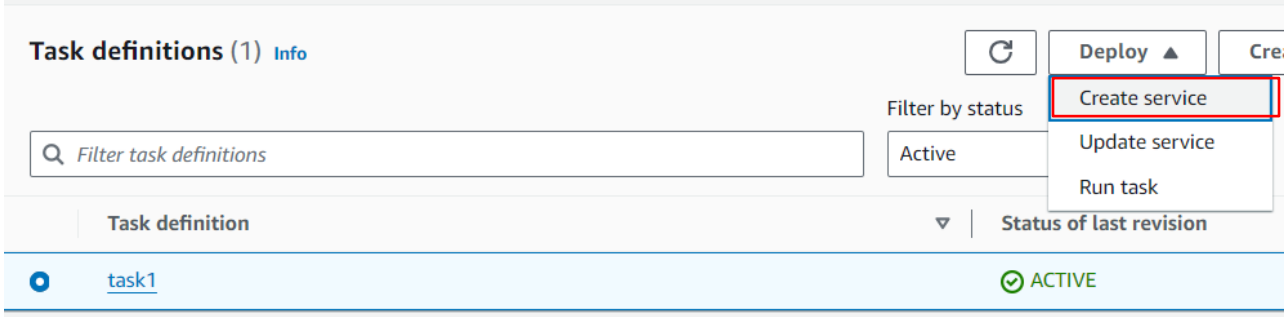


Step 13 :- Enter the container details

- Container name:- myc1
- image url :-copy the URI from the Repo that we created above.
- Port mapping :- 80
- Port name :- httpd



Step 14:- Go back and under the deploy section , choose create service for our task



- Choose Cluster
- Give the service name

Create Info

Environment

AWS Fargate

Existing cluster

myCluster1 | Create a new cluster

▼ Compute configuration *(advanced)*

Compute options Info

To ensure task distribution across your compute types, use appropriate compute options.

- (●) **Capacity provider strategy** — Specify a launch strategy to distribute your tasks across one or more capacity providers.
- ( ) **Launch type** — Launch tasks directly without the use of a capacity provider strategy.

Capacity provider strategy Info

Select either your cluster default capacity provider strategy or select the custom option to configure a different strategy.

- ( ) Use cluster default
  No default capacity provider strategy configured for this cluster.
- (●) Use custom (Advanced)

Capacity provider	Base Info	Weight Info
FARGATE	0	1

Add capacity provider

Platform version Info

Specify the platform version on which to run your service.

LATEST

Deployment configuration

Application type Info

Specify what type of application you want to run.

- (●) **Service** — Launch a group of tasks handling a long-running computing work that can be stopped and restarted. For example, a web application.
- ( ) **Task** — Launch a standalone task that runs and terminates. For example, a batch job.

Task application type is not applicable when launching a service.

Task definition

Select an existing task definition. To create a new task definition, go to Task definitions.

☐ Specify the revision manually

Manually input the revision instead of choosing from the 100 most recent revisions for the selected task definition family.

Family	Revision
task1	1 (LATEST)

Service name

Assign a unique name for this service.

myservice1

Service type Info

Specify the service type that the service scheduler will follow.

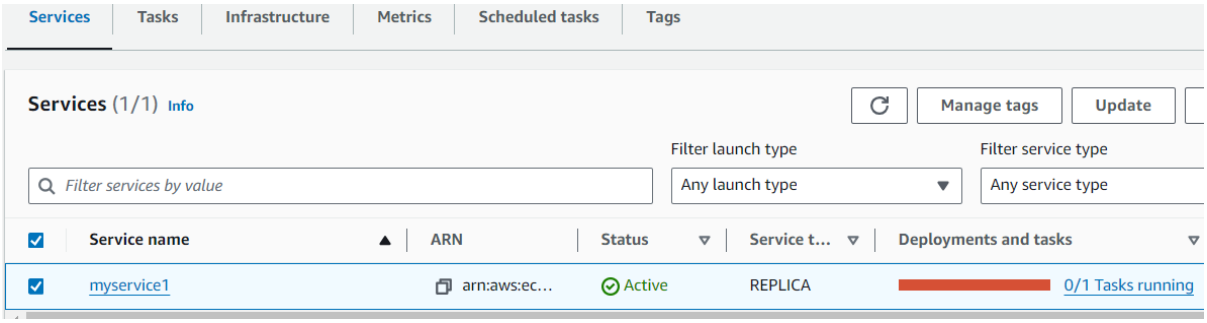
- (●) **Replica** — Place and maintain a desired number of tasks across your cluster.
- ( ) **Daemon** — Place and maintain one copy of your task on each container instance.

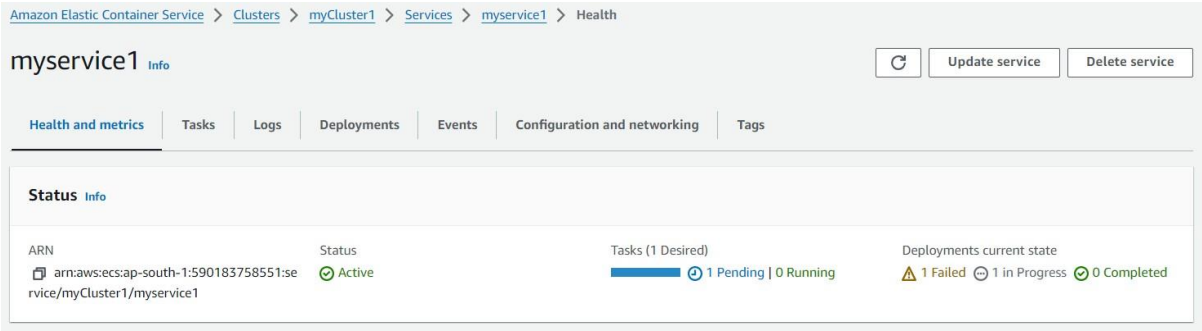
Desired tasks

Specify the number of tasks to launch.

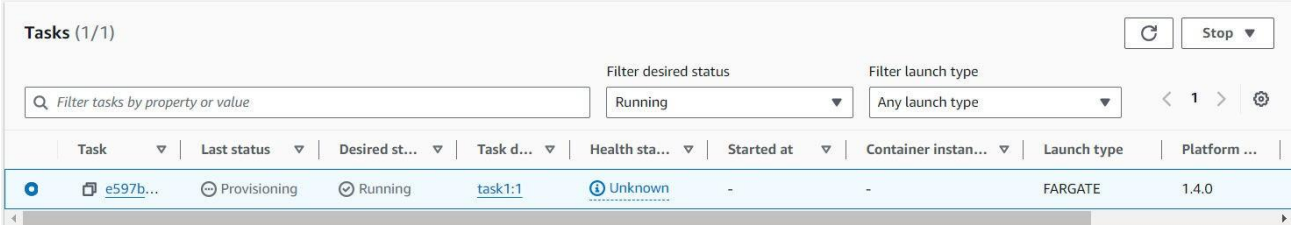
1

PRATHAMESH MISAL

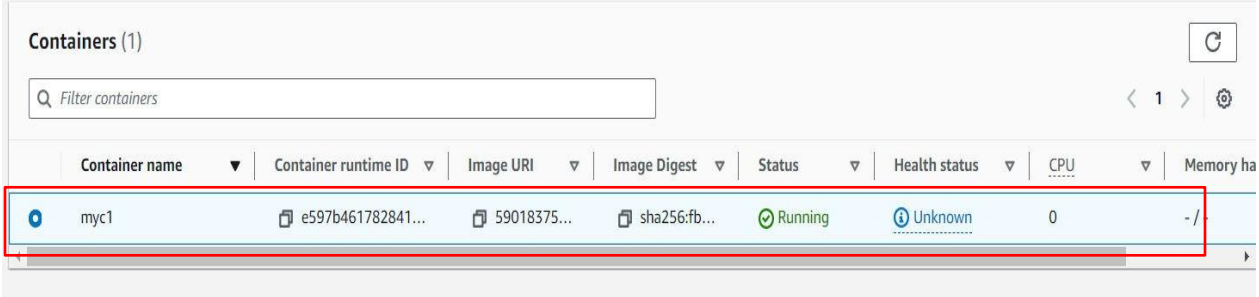




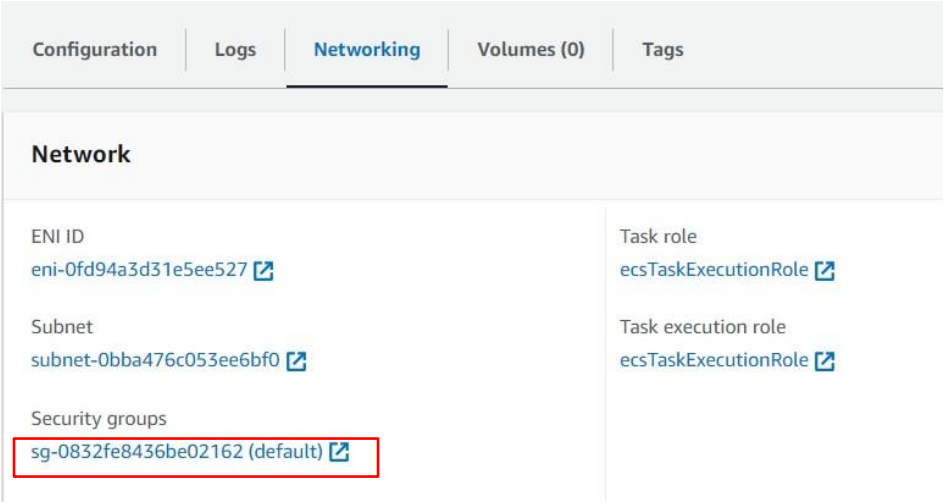
- Go to the tasks section



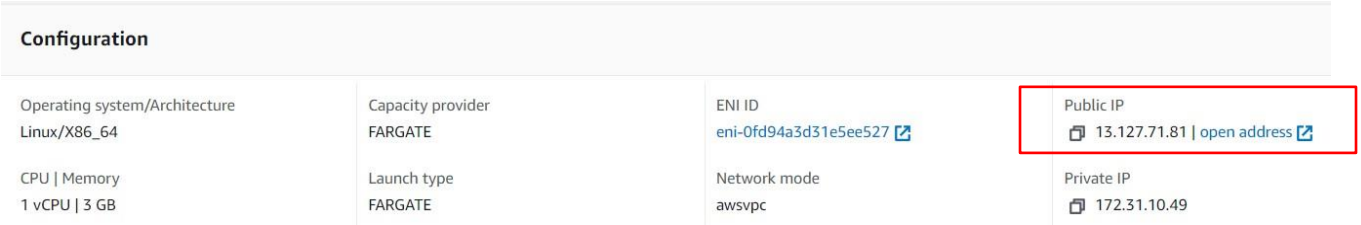
- Click on the above task and we can see our container is now running



- Go inside networking section and make sure to add port 80 to our security group that is attached



- Open the Public ip



- We can now able to access our Container